13 ESEMPIO GUIDATO

Questo capitolo applica molte delle lezioni trattate nel libro alla costruzione di una soluzione software di esempio. Si parte dalla descrizione iniziale del problema fino al testing del prodotto finito. La soluzione presentata è semplificata ed è pensata per evidenziare determinati concetti. Alla fine del capitolo vengono discusse possibili migliorie per renderla più sofisticata.

DEFINIZIONE DEL PROBLEMA

Progettare un sistema di automazione domestica controllato da computer. Il sistema deve gestire le seguenti funzioni della casa:

Ventilazione

- Regola il livello di umidità nell'aria, che non deve superare il **70%**.
- Introduce regolarmente aria esterna nella casa.
- Deve funzionare a orari programmati.

Riscaldamento

- Ogni stanza ha un **radiatore** che può essere acceso o spento per regolare la temperatura.
- La temperatura ideale è **22°C**.
- La temperatura non deve scendere sotto i **18°C**.

Illuminazione

- Le luci devono accendersi automaticamente quando la stanza è occupata e spegnersi quando la stanza è vuota.

Pannello di controllo

Il sistema deve poter essere gestito da un pannello di controllo centrale con le seguenti funzioni:

- Programmare gli orari della ventilazione.
- Configurare la temperatura minima e ottimale del riscaldamento.

PENSIERO COMPUTAZIONALE

DECOMPOSIZIONE DEL PROBLEMA

Scomponiamo il problema in parti più piccole e affrontabili separatamente.

Il sistema riguarda **automazione e controllo utente**. Ogni sottosistema **agisce automaticamente** o permette all'utente di configurarlo.

Primo livello di scomposizione:

```
Definizione del problema
 ├── Automazione
 └── Controllo
```

Il secondo livello suddivide queste due categorie tra i **tre sottosistemi principali**:

```
Definizione del problema
 ├── Automazione
 │   ├── Ventilazione
 │   ├── Illuminazione
 │   └── Riscaldamento
 └── Controllo
     ├── Ventilazione
     ├── Illuminazione
     └── Riscaldamento
```

Successivamente, analizziamo le azioni specifiche di ciascun sottosistema.

Esempio per l'illuminazione:

- Misurare l'occupazione della stanza.
- Accendere/spegnere le luci in base alla presenza delle persone.

Ora i sotto-problemi sono più chiari e gestibili singolarmente.

TROVARE PATTERN

Dalla decomposizione possiamo identificare schemi ripetuti.

Entità nel sistema

- **Stanze**
- **Pannello di controllo**
- **Sottosistemi**:
    - Riscaldamento
    - Illuminazione
    - Ventilazione
- **Componenti**:
    - Radiatori
    - Luci
    - Ventilatori
- **Sensori**:
    - Sensore di temperatura
    - Sensore di occupazione
    - Sensore di umidità

Possiamo generalizzare molte di queste entità in **sottosistemi, componenti e sensori**.

Azioni nel sistema

1. **Rilevare misurazioni** (temperatura, umidità, occupazione, orario).
2. **Reagire ai cambiamenti ambientali** (accendere/spegnere i componenti).
3. **Controllo manuale** (attivare/disattivare il sistema, impostare parametri).

Possiamo semplificare molte di queste azioni in **gruppi generali**.

GENERALIZZAZIONI E ASTRAZIONI

Riassumendo la scomposizione:

- Il sistema ha **tre sottosistemi** e un **pannello di controllo**.
- Ogni sottosistema ha **sensori** e **componenti**.
- Ogni sensore è collocato in una stanza.
- Ogni sottosistema monitora i sensori e attiva i componenti in base a regole predefinite.

Esempio di regole

- Se la temperatura è inferiore a **18°C**, il radiatore si accende.

- Se la temperatura supera il valore ottimale, il radiatore si spegne.
- Se una stanza è occupata, le luci si accendono.
- Se l'umidità supera il **70%**, la ventilazione si attiva.

MODELLI UML

Per chiarire il funzionamento, usiamo diagrammi UML.

Classi principali

1. **Regola** (classe base per tutte le regole).
2. **Sensore** (classe base per tutti i sensori).
3. **Componente** (classe base per radiatori, luci, ventilatori).
4. **Regolatore** (gestisce le regole e i componenti associati).

CODICE SORGENTE

Il codice è organizzato in pacchetti Python:

markdown

CopiaModifica

```
automation_system/
 ├── core/
│   ├── __init__.py
│   ├── parts.py
│   ├── regulator.py
 ├── subsystems/
│   ├── __init__.py
│   ├── heating.py
│   ├── lighting.py
│   ├── ventilation.py
```

Esempio di classe TemperatureRule

python

CopiaModifica

```
class TemperatureRule(Rule):
  def apply(self, reading, radiator):
    if reading < 18:
      radiator.switch_on()
    elif reading > 22:
      radiator.switch_off()
```

TESTING

Unit test per l'illuminazione

python

CopiaModifica

```
import unittest
import subsystems.lighting as light

class TestLightingRule(unittest.TestCase):
  def setUp(self):
    self.light_rule = light.LightingRule()
    self.light = light.Light(1)

  def test_lights_come_on(self):
    self.light_rule.apply(True, self.light)
    self.assertFalse(self.light.off)

if __name__ == '__main__':
  unittest.main()
```

📌 **Esegui il test con:**

sh

CopiaModifica

python -m unittest test/test_lighting.py

POSSIBILI MIGLIORAMENTI

- **Aggiungere il codice per il pannello di controllo.**
- **Regolatori più sofisticati**, che supportano più regole contemporaneamente.
- **Componenti avanzati**, come luci con intensità regolabile.
- **Migliorare il countdown dell'illuminazione**, per evitare spegnimenti fastidiosi.
- **Scrivere un test di sistema che simula il funzionamento simultaneo dei tre sottosistemi.**

CONCLUSIONE

Questo esempio ha dimostrato come applicare la **decomposizione**, il **pattern recognition**, la **modellizzazione** e il **testing** per sviluppare una soluzione software.

📌 **Esercizio finale** → Prova a migliorare il codice o ad aggiungere nuove funzionalità!